Focus Noise Sculptor — v1 Build Plan

A procedural white/pink/brown noise sculptor for iOS, modeled on the myNoise white-noise generator. v1 ships pure synthesized noise with a frequency-band slider bank, animation, presets, and a focus-session layer. Architected so nature layers (rain, etc.) are a plug-in addition later, not a rewrite.

---

1. Product definition (lock this before building)

One-sentence wedge: A focus app that lets you sculpt your own noise across the hearing spectrum and have it gently evolve, instead of picking a static track, beating both Apple's free Background Sounds and generic loop players.

v1 scope (in):

- Procedural white / pink / brown / grey noise, generated in real time.
- A multi-band frequency slider bank (the core interaction).
- Built-in presets (Brown, Pink, White, Grey, Speech Blocker) as saved slider positions.
- Animation engine (gentle auto-modulation of bands so it never feels static).
- Stereo width control.
- User-saved custom presets (local + iCloud).
- Focus session layer: timer (deep-work + Pomodoro), Live Activity, gentle end chime.
- Background playback, interruption/route handling.

v1 scope (out, deliberately deferred):

- Rain / nature / ambient recorded layers (future hybrid).
- Hearing calibration feature (strong moat, but post-v1).
- Watch app.
- Habit stats/streaks (add once retention is proven).
- Account system / login (use iCloud, no auth in v1).

Out-of-scope discipline: if a feature isn't in the "in" list, it waits. The risk in this category is over-building before proving retention.

---

2. Audio engine architecture

This is the hard, defensible core. Build and validate this first.

2.1 The layer model (the abstraction that protects the hybrid)

Everything the app plays is a Layer. A layer has a type; v1 ships only the procedural-noise type, but the model already supports more.

```swift
enum LayerSource {
```

```
    case proceduralNoise(NoiseParams)   // v1
    case audioFile(URL)                 // future (rain, etc.) — stubbed, not built
}

struct Layer: Codable, Identifiable {
    let id: UUID
    var name: String
    var source: LayerSource
    var volume: Float          // 0...1
    var isEnabled: Bool
}

struct NoiseParams: Codable {
    var bandGains: [Float]     // one entry per EQ band (the sliders)
    var stereoWidth: Float     // 0 = mono ... 1 = wide
    var animation: AnimationParams?
}
```

The mixer iterates over [Layer] and asks each for an audio node. It never branches on "noise vs file" at the app level. Adding rain later = adding the audioFile case's node-building code; nothing above it changes.

2.2 The node graph (AVAudioEngine)

```
[per-layer source nodes] !' [per-layer mixer/volume] !' [main mixer] !' [o
```

- Procedural noise source: an AVAudioSourceNode whose render callback fills the buffer with white noise (uniform random samples, or summed for a smoother distribution). This is the generator. Zero audio files, near-zero app size, no loop seams.
- The band bank (the sliders): route the noise through an AVAudioUnitEQ configured with N bands (start with the myNoise set: ~60Hz, 125, 250, 500, 1k, 2k, 4k, 8k). Each UI slider maps directly to one band's gain. This is the sculptor.
- White = all bands flat. Pink "H -3 dB/octave tilt. Brown "H -6 dB/oc ear's sensitivity curve. So your presets are just band-gain arrays.
- Stereo width: generate decorrelated left/right noise (separate random streams) and adjust per-channel balance; width control blends between mono (correlated) and wide (decorrelated).
- Main mixer !' output: standard AVAudioMixerNode to the engine's

2.3 The animation engine

What keeps it from sounding static and dead. A low-frequency modulation routine nudges band gains over time.

- A timer (or a per-render update at low rate) slowly moves selected band gains around their set value.
- Params (mirror myNoise): mode (soft/hard = modulation depth), voices (solo/duo/trio/quad = how many bands move at once), speed (÷8 … x8), range (which part of the spectrum animates).
- Implementation: each animated band gets an LFO with randomized phase; gain = setValue + depth × lfo(t). Keep motion slow and smooth or it sounds robotic.
- This is cheap to build once the band bank exists, which is why band bank comes first.

2.4 Session & playback infrastructure (shared by all future layer types)

- Configure AVAudioSession category .playback, declare the audio background mode in capabilities.
- Handle interruptions (calls, Siri, other audio) and route changes (headphones unplugged → stop, don't blast speakers) via AVAudioSession notifications.
- Test battery on real devices: continuous synthesis + Live Activity has a cost; budget and measure.

2.5 Preset format (layer-agnostic from day one)

```swift
struct Preset: Codable, Identifiable {
    let id: UUID
    var name: String
    var layers: [Layer]        // a list, not "16 slider values"
    var createdAt: Date
}
```

A preset is a list of layers. Today every preset has one noise layer. Tomorrow a preset can have noise + rain with no format migration. Persist locally; sync via iCloud (NSUbiquitousKeyValueStore or a small CloudKit record). Optional share-via-URL later (encode preset into a deep link, like myNoise's "Save to URL").

---

3. Screen flow (v1)

Keep it minimal. The point is to start focusing, not to fiddle.

- Home / Sculptor (single primary screen)
- The band slider bank, front and center (vertical sliders, one per frequency band).
- Preset row along top or bottom: Brown / Pink / White / Grey / Speech Blocker + user presets.
- Big Play/Pause.
- Secondary controls (tap to reveal, don't clutter): stereo width, animation toggle + params.
- "Save preset" action when the user has tweaked sliders.

- Focus session (sheet or mode on top of sculptor)
- Choose duration: deep-work (set minutes) or Pomodoro (work/break cycles).

- Start ’ Live Activity on lock screen ’ gentle end chime.
- No jarring stops; fade out.

- Presets management (light)
- List of saved presets, rename/delete, reorder.

That's it for v1. Three surfaces.

---

4. Monetization sequencing (long-term goal, light touch in v1)

- v1: ship free, no paywall. First prove the engine sounds great and that people retain. A paywall on an unproven app just adds friction to your learning.
- v1.x: introduce freemium once retention is real.
- Free: white/pink/brown sliders, basic timer, a few saved presets.
- Paid (subscription, annual-forward, ~$3–5/mo billed yearly): animation engine, stereo controls, unlimited presets, iCloud sync, and (when built) calibration + nature layers.
- Subscription over one-time purchase: the category norm, and it funds ongoing content (nature layers) and audio work later.
- myNoise itself gates animation-adjacent and extra features behind patron support, a working freemium signal for this exact niche.

---

5. Build roadmap (sequenced to de-risk the hard part first)

Phase 0 — Audio spike (prove it sounds good):

- AVAudioEngine + AVAudioSourceNode generating white noise.
- One AVAudioUnitEQ band bank wired to a handful of test sliders.
- Validate: clean sound, no artifacts, white/pink/brown achievable by tilting bands. Gate: if synthesized noise doesn't sound good enough, stop and rethink before building UI.

Phase 1 — Core sculptor:

- Full band bank + slider UI on the Home screen.
- Built-in presets as band-gain arrays.
- Background mode, audio session, interruption + route handling.
- Play/pause, fade in/out.

Phase 2 — Make it alive + savable:

- Animation engine + its controls.
- Stereo width.
- Save/load custom presets (layer-agnostic format) + iCloud sync.

Phase 3 — Focus session layer:

- Timer (deep-work + Pomodoro), Live Activity, end chime.

Phase 4 — Ship + measure:

- TestFlight, then App Store.
- Instrument 30-day retention as the make-or-break metric (target beating ~10–15%).

Later (not v1): freemium paywall, calibration, Watch app, habit stats, and the rain/nature hybrid (new layer type bolted onto the existing engine).

---

6. Key risks & what would change the plan

- Synthesized noise quality: if it sounds thin/digital vs recorded noise, Phase 0 surfaces it early. Mitigation: proper noise distribution, careful EQ, subtle animation.
- Retention, not installs: novelty installs are easy; churn is savage. If 30-day retention is poor even with a great engine, the monetization thesis is wrong and you rethink the wedge, not the code.
- Battery complaints: continuous synthesis + Live Activity. Measure on real devices in Phase 1, not at the end.
- Audio licensing: irrelevant in v1 (you generate everything), becomes real only when you add nature layers. A point in favor of shipping Path B first.

---

7. Immediate next action

Build Phase 0 (the audio spike). Everything else is wasted effort if synthesized noise + an EQ band bank doesn't sound clean and pleasant on real headphones. Prove that first, then build the UI around it.